

Axios

- Axios is a promise-based HTTP Client for node.js and the browser. On the server-side it uses the native node.js http module, while on the client (browser) it uses XMLHttpRequests.
- Axios is a Javascript library that allows you to connect with the backend API and manage requests made via the HTTP protocol.
- It also provides a more flexible and powerful feature set.
- It is similar to the native JavaScript Fetch API Method.

Prerequisites

Let's consider some prerequisites before proceeding.

- Familiar with yarn or node npm package module.
- Create react app tool pre-installed.
- Basic knowledge of HTML/CSS and Javascript(ES6).
- Basic understanding of command-line terminal.

Installing Axios

- In order to use Axios with React, we need to install Axios. It does not come as a native JavaScript API, so that's why we have to manually import into our project.
- There are many ways to install Axios. You can pick any of them based on your system environment.
- Open up a new terminal window, move to your project's root directory, and run any of the following commands to add Axios to your project.

- **Using npm:** `$ npm install axios`
- **Using bower package manager:** `$ bower install axios`
- **Using yarn:** `$ yarn add axios`
- **Using unpkg CDN:** `<script
src="https://unpkg.com/axios/dist/axios.min.js"></script>`
- **Using jsDelivr CDN:** `<script
src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>`

Features

- Make XMLHttpRequests from the browser
- Make http requests from node.js
- Supports the Promise API
- Intercept request and response
- Transform request and response data
- Cancel requests
- Automatic transforms for JSON data
- Client side support for protecting against XSRF

Looking at the response object

When we send a request to server and it is successful, our then() callback will receive a response object which can have the following properties:

- **data:** It is the payload returned from the server. By default, Axios expects JSON and will parse this back into a JavaScript object for you.
- **status:** It is basically the HTTP code returned from the server.
- **statusText:** it refers to the HTTP status message returned by the server.
- **headers:** It contains all the headers sent back by the server.
- **config:** It has the original request configuration.
- **request:** It is the actual XMLHttpRequest object (when it is running in a browser)

Looking at the error object

And if there's a problem with the request, the promise will be rejected with an error object containing at least one or few of the following properties:

- **message:** the error message text.
- **response:** the response object (if received) as described in the previous section.
- **request:** the actual XMLHttpRequest object (when running in a browser).
- **config:** the original request configuration.

Axios status code

HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

- Informational responses (100–199)
- Successful responses (200–299)
- Redirects (300–399)
- Client errors (400–499)
- Server errors (500–599)

Shorthand Methods in Axios

You have already seen that Axios provides an easy-to-use API contract model in a compact package for most of our HTTP communication needs.

Do you know that Axios has some shorthand methods for performing different HTTP requests?

Check them below:

- `axios.request(config)`
- `axios.get(url[, config])`
- `axios.delete(url[, config])`
- `axios.head(url[, config])`
- `axios.options(url[, config])`
- `axios.post(url[, data[, config]])`
- `axios.put(url[, data[, config]])`
- `axios.patch(url[, data[, config]])`

Axios :-

PROS:

- Promise based HTTP client for the browser and server (NodeJS)
- Compatible with some older browsers (ie. IE11)
- Provides Cancelling Request $\Rightarrow$ that can also be done without Axios using AbortController)
- Very popular third-party library actively supported
- Wrapper / decorator pattern which offers nice and convenient interface

CONS:

- Must be installed and imported (not native in JavaScript)
- Not the standard so it is important to manage the conflict possibilities
- Third-party libraries adds weight/load on the application or website (to be considered)

Difference B/w Axios and Fetch

- Axios
- Axios has **url** in request object.
- Axios is a **stand-alone third party package** that can be easily installed.
- Axios enjoys built-in XSRF protection.
- Axios uses the **data** property.
- Axios' data contains the **object**.
- Axios request is ok when **status is 200** and **statusText is 'OK'**.
- Axios performs **automatic transforms of JSON data**.
- Axios allows **cancelling request and request timeout**.
- Axios has the ability to **intercept HTTP requests**.
- Axios has **built-in support for download progress**.
- Axios has **wide browser support**.
- Fetch
- Fetch has **no url** in request object.
- Fetch is built into most modern browsers; **no installation** is required as such.
- Fetch does not.
- Fetch uses the **body** property.
- Fetch's body has to be **stringified**.
- Fetch request is ok when **response object contains the ok property**.
- Fetch is a **two-step process** when handling JSON data- first, to make the actual request; second, to call the `.json()` method on the response.
- Fetch does not.
- Fetch, by default, doesn't provide a way to intercept requests.
- Fetch does not support upload progress.
- Fetch only supports Chrome 42+, Firefox 39+, Edge 14+, and Safari 10.1+ (This is known as Backward Compatibility).

Simple GET request - Axios vs Fetch

Axios GET request:

1. `// Simple GET request using axios`
2. `Const element = document.querySelector('#get-request .axios.total');`
3. `axios.get('https://reqres.in/api/users')`
4. `.then(response => element.innerHTML = response.data.total);`

Fetch GET request

1. `// Simple GET request using fetch`
2. `const element = document.querySelector('#get-request .fetch .total');`
3. `fetch('https://reqres.in/api/users')`
4. `.then(response => response.json())`
5. `.then(data => element.innerHTML = data.total );`